

Modelos sequenciais e o nascimento da atenção

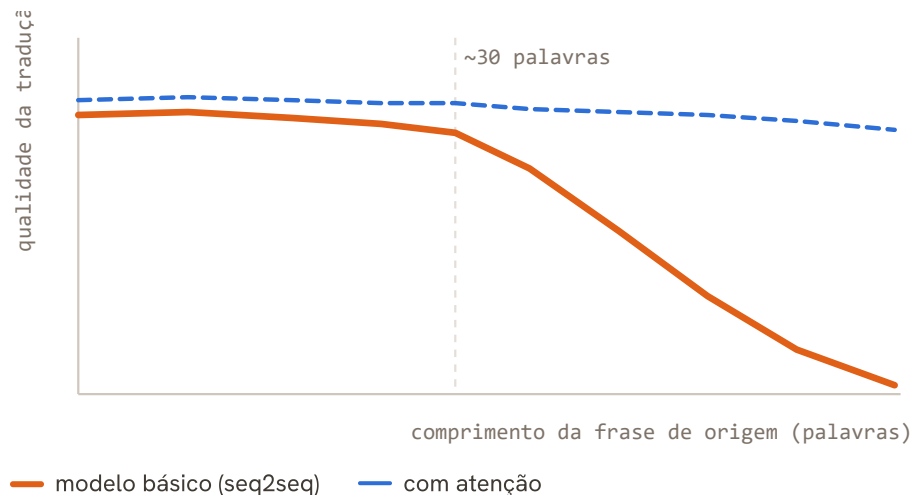
Do comportamento observável ao mecanismo: três dívidas da recorrência e a troca de compressão por acesso.

Nome e Sobrenome

seuemail@cesar.org.br

Agosto / 2026

Acerta a manchete, erra o parágrafo



3 PALAVRAS

"o gato dorme" → *the cat sleeps*

40 PALAVRAS

...o relatório que os pesquisadores entregaram [30 palavras] **was** aprovado

O erro não é aleatório: ele acompanha o **comprimento da entrada**. Isso torna o gargalo arquitetural uma hipótese forte — e mesmos dados, mesmo protocolo, decidem.

TESE Uma curva não fecha o diagnóstico. Ela aponta a suspeita — e o experimento comparativo decide.

• COMPARAÇÃO

“Então é só aumentar o vetor”

Aumentar o resumo interno compra **espaço**, não **acesso**: o decoder continua recebendo um único vetor e não pode consultar novamente as posições da entrada.

CENÁRIO A

Prova de memória

Olhos fechados. O que valer, vale do que sobrou na cabeça. Um caderno maior não muda a regra.

fonte indisponível

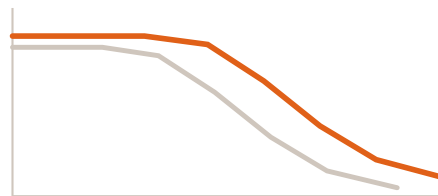
CENÁRIO B

Prova com consulta

Livro aberto. Cada resposta pode voltar à fonte e escolher onde olhar.

fonte consultável

AUMENTAR O VETOR FAZ ISTO

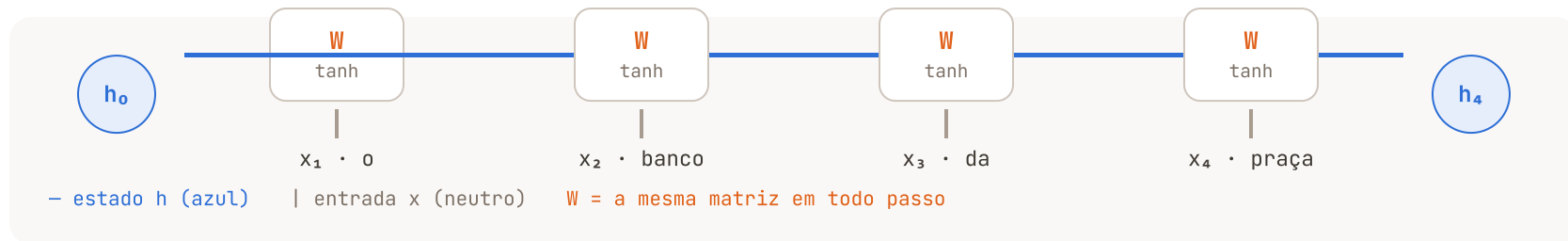


A mesma forma, deslocada. A queda continua lá.

TESE Falta acesso, não espaço. Comprar um caderno maior não resolve um problema de não poder olhar a fonte.

• CONCEITO EM USO

Uma função, aplicada em loop



$$h_t = f(h_{t-1}, x_t)$$
$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

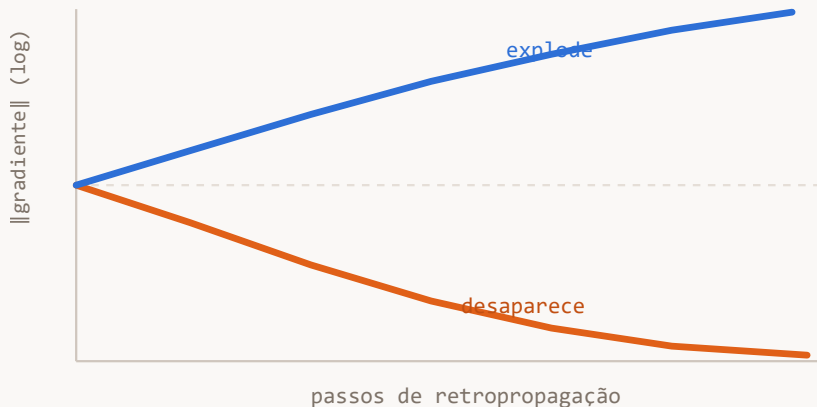
- h_t é o resumo comprimido de tudo o que veio antes.
- Mesma matriz em todo passo $\rightarrow$ n° de parâmetros independe de T .
- A ordem entra de graça — e os tokens não são guardados.

TESE É um reduce sobre a frase, com a função de combinação aprendida.

[→ Apêndice A.1](#)

Concorda a três palavras, erra a quarenta

A dependência longa **nunca** entra. Sem NaN, sem divergência, com a loss descendo — o sinal de treino chega atenuado aos primeiros passos.



NA CALCULADORA, À VISTA DA TURMA

$0,9^{10}$

0,35

$1,1^{50}$

117

$0,9^{50}$

$5 \cdot 10^{-3}$

$1,1^{100}$

$1,4 \cdot 10^4$

$0,9^{100}$

$3 \cdot 10^{-5}$

Não é falta de capacidade: mantido o ganho por passo, o expoente T continua lá.

TESE O modelo pode ter capacidade para a dependência e ainda não aprendê-la.

→ [Apêndice A.2](#)

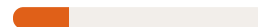
Trocaram a GPU por uma quatro vezes mais rápida e ganharam 15%

A causa é estrutural: h_t exige $h_{(t-1)}$. São T etapas obrigatoriamente sequenciais — e a dívida é de **treino**, não de geração.

RECORRÊNCIA
em série



OCUPAÇÃO DA GPU



SEM RECORRÊNCIA
Aula 6



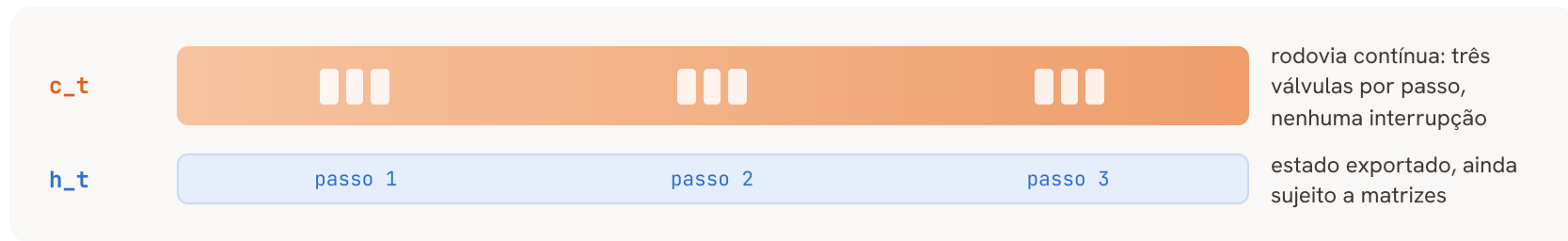
OCUPAÇÃO DA GPU



tempo → paralelizar o treino **paga em memória** — o custo que a Aula 6 apresenta.

TESE O que matou a recorrência não foi qualidade. Foi ela não deixar você gastar dinheiro para ir mais rápido.

Tudo o que importa está num sinal de mais



```
ft, it, ot = o(W · [h(t-1), xt])
```

três válvulas entre 0 e 1

```
ct = ft ⊙ c(t-1) + it ⊙ gt
```

o caminho ADITIVO

Ganha-se: dependência de médio alcance aprendível, treino estável.

Não se ganha: quando as válvulas fecham, o decaimento volta — o gating põe o desaparecimento *sob controle da rede*.

O **ELMo** (2018) contextualizou banco com LSTMs bidirecionais, **sem atenção**. Contextualizar, consultar e paralelizar são operações distintas.

TESE O LSTM não resolveu o gradiente. Ele deu à rede o poder de decidir quando não estragá-lo.

→ [Apêndice A.3](#)

Aliviada, contida, intacta

LIMITAÇÃO	RNN SIMPLES	LSTM (1997)	BAHDANAU • TRANSFORMER
Memória distante e caminho do gradiente	quebra	aliviada 499 passos continuam lá	acesso em 1 passo aos estados do encoder
Contexto fixo um vetor por frase	gargalo	contido recorte segura a explosão	resolve acesso — e continua recorrente
Ausência de paralelismo h _t exige h _(t-1)	estrutural	intacta nada mudou	só o Transformer paraleliza o treino

TESE Duas dívidas foram renegociadas. A terceira não foi nem discutida — e é ela que quebra a arquitetura.

• DIAGRAMA

Um único ponto de falha, e ele tem nome

O encoder lê a origem inteira e só atualiza estado; o decoder parte do último estado e escreve.

Daí em diante, o decoder **não tem mais acesso à frase de origem**.



TESE A curva da abertura é o retrato de um vetor de tamanho fixo carregando uma entrada de tamanho variável.

→ [Apêndice A.4](#)

Três medições, doze minutos, nenhuma biblioteca de deep learning

01 o produto que decide o gradiente

demo-gradiente-rnn.py · norma para $T = 10, 50, 200$

02 a compressão em quatro números

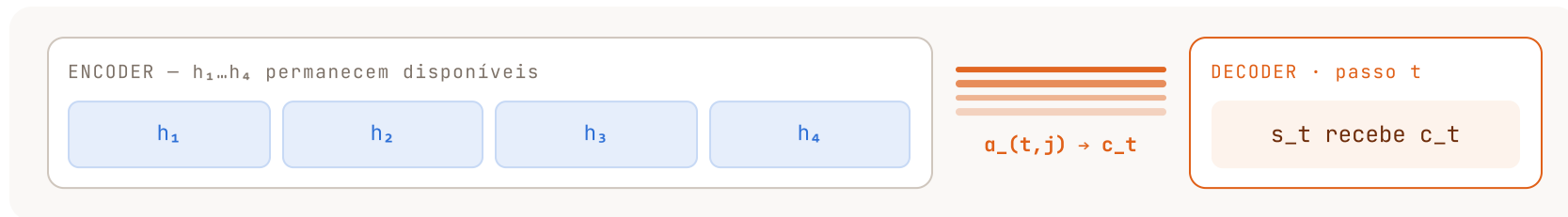
no quadro, com a turma — a frase é apagada

03 a curva e a matriz

Figuras 2 e 3 de Bahdanau et al. (1409.0473)

TESE Nenhuma das três é ilustração. As três são a evidência dos slides anteriores.

Um vetor de contexto por passo, não um por frase



$$\begin{aligned} e_t(t, j) &= v^T \tanh(W s_{t-1} + U h_j) \\ a_t(t, j) &= \exp(e_t(t, j)) / \sum_k \exp(e_t(t, k)) \\ c_t &= \sum_j a_t(t, j) h_j \\ a_t &= (0,6; 0,3; 0,1) \rightarrow c_t = 0,6h_1 + 0,3h_2 + 0,1h_3 \end{aligned}$$

compara o estado do decoder com cada posição da origem;
vira orçamento que soma 1;

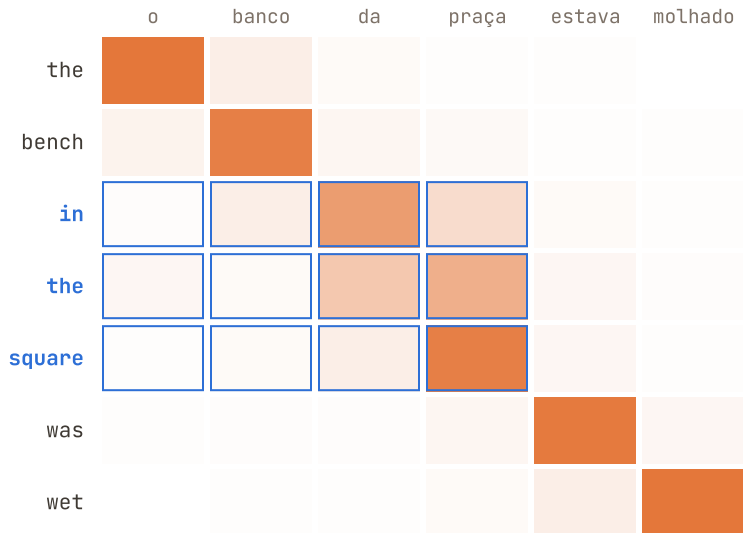
lê a origem segundo esse orçamento — caminho de comprimento **1**.

Em 2014 a atenção **não** substitui a recorrência: encoder e decoder continuam RNNs.

TESE O conteúdo é o mesmo; o que muda é o direito de olhar a fonte na hora de responder.

→ **Apêndice A.5**

O que o modelo aprendeu sem rótulo



linhas = passos do decoder • colunas = posições do encoder • cada linha soma 1 • fig. 3, Bahdanau et al., arXiv 1409.0473

A diagonal predominante mostra que as ordens das duas línguas coincidem na maior parte. **O bloco marcado** é onde ela se rompe: um grupo nominal cuja ordem inverte.

Nenhum rótulo de alinhamento foi usado no treino.

O único sinal foi a tradução correta. O alinhamento apareceu porque era o caminho mais barato para acertar.

Peso alto é correlação com a decisão, não a razão dela — por isso os autores dizem alinhamento *suave*.

TESE Ninguém ensinou alinhamento a esse modelo. Ele foi treinado para traduzir.

A reescrita que abre a próxima aula

O que a atenção faz tem a forma de uma **busca**: há uma consulta, há itens indexados, há conteúdo devolvido.

CONSULTA · Q

$$q_t = W_s s_{(t-1)}$$

deriva do estado do decoder — $s_{(t-1)}$

CHAVE · K

$$k_j = U_h h_j$$

deriva de h_j e **atravessa** U_h no score

VALOR · V

$$v_j = h_j$$

entra cru na soma ponderada — logo $K \neq V$

Duas diferenças em relação a um dict: a comparação não é igualdade exata (é uma função **aprendida** que devolve um número contínuo) e o resultado não é um item, é uma **mistura** ponderada de todos.

AULA 6

$$Q = X W_Q \cdot K = X W_K \cdot V = X W_V$$

e as queries vêm da própria sequência

TESE Q, K e V começam como três papéis. No Transformer, viram três projeções explícitas da mesma entrada.

→ [Apêndice A.6](#)

Diagnosticar, não calcular

• 08:00

Para cada situação: qual é a causa provável e **que evidência confirmaria**.

01

Um tradutor acerta manchetes e desmonta em parágrafos. O erro cresce com o comprimento; a primeira coisa a quebrar é a concordância entre partes distantes.

CAUSA

EVIDÊNCIA QUE CONFIRMA

02

Um modelo recorrente acerta a concordância entre vizinhas e **nunca** aprende a de quarenta posições. A loss desce, não há NaN, e aumentar a dimensão do estado não mudou nada.

CAUSA

EVIDÊNCIA QUE CONFIRMA

03

Um time trocou a GPU por outra quatro vezes mais rápida e o tempo por época caiu 15%. Mesmo modelo, mesmos dados, mesmo tamanho de lote.

CAUSA

EVIDÊNCIA QUE CONFIRMA

TESE Nenhuma das três se resolve calculando. As três se resolvem sabendo o que a arquitetura descarta.

● ENCERRAMENTO

uma curva que cai com o comprimento



ler em ordem deixa três dívidas



um vetor fixo agrava memória e acesso



atenção troca compressão por ACESSO

Se a atenção dá acesso direto, para que ainda serve a recorrência?



2017 • revelado na Aula 6

Leitura da semana: Bahdanau, Cho & Bengio (arXiv 1409.0473), seções 1-3 e a discussão da Figura 3.

Pergunta dirigida: por que a Figura 3 é evidência de alinhamento aprendido sem rótulo de alinhamento?

Aula 6 – Transformer I:
self-attention e a arquitetura

Apêndice matemático

Reconstrói, com rigor completo, todo fundamento citado no fluxo.

Autossuficiente: quem estuda por aqui sem ter assistido à aula consegue reconstruir tudo.

A.1 · a recorrência

A.2 · gradiente em t

A.3 · gating do LSTM

A.4 · vetor fixo

A.5 · alinhamento

A.6 · para Q/K/V

A.1 A recorrência, escrita e desenrolada

invocado no slide 03

NOTAÇÃO E PREMISSAS

$x_t \in \mathbb{R}^{(d_x)}$ · entrada no passo t

$h_t \in \mathbb{R}^{(d_h)}$ · estado; $h_0 = 0$

W_h, W_x, b · os mesmos em todo passo

T · comprimento da sequência

Não linearidade tanh, componente a componente. O argumento vale para qualquer f diferenciável com derivada limitada; a cota da derivada só é usada em A.2.

FORMA GERAL E INSTÂNCIA

```
h_t = f(h_{(t-1)}, x_t)
h_t = tanh(W_h h_{(t-1)} + W_x x_t + b)
y_t = W_y h_t + b_y
```

ETAPA 1 · DESENROLAMENTO ($T = 3$)

```
h_1 = tanh(W_h h_0 + W_x x_1 + b)
h_2 = tanh(W_h h_1 + W_x x_2 + b)
h_3 = tanh(W_h tanh(W_h tanh(...) + W_x x_2 + b) + W_x x_3 + b)
```

É uma rede profunda de T camadas que **compartilha os pesos** — o mesmo W_h multiplica em todos os níveis.

ETAPA 2 · CONTAGEM DE PARÂMETROS

$$|W_h| + |W_x| + |b| = d_h^2 + d_h \cdot d_x + d_h$$

Nenhum termo depende de T — daí processar qualquer comprimento. O preço é reaplicar a mesma transformação, que A.2 transforma em problema.

ETAPA 3 · POR QUE A ORDEM IMPORTA

Composição de funções não comuta: $f(f(h, x_a), x_b) \neq f(f(h, x_b), x_a)$. Com $d_h = d_x = 1$, $W_h = W_x = 1$, $b = 0$, $h = 0$, $x_a = 1$, $x_b = 2$:

$$(a, b): h_1 = 0,7616 \rightarrow h_2 = \tanh(2,7616) = \mathbf{0,9922}$$

$$(b, a): h_1 = 0,9640 \rightarrow h_2 = \tanh(1,9640) = \mathbf{0,9600}$$

Os estados finais diferem. ■ A self-attention, ao contrário, **comuta** com a permutação — a posição precisa ser injetada de fora (Aula 7).

CASOS-LIMITE

$W_h = 0 \rightarrow$ sem memória; a ordem deixa de importar.

f linear com $W_h = I \rightarrow h_t = \sum W_x x_s$: memória perfeita, sem seletividade.

$d_h = 1 \rightarrow$ todo o passado num escalar; o limite de A.4 aparece no primeiro token.

Intuição. Um *reduce* sobre a frase: h é o acumulador, x_t o elemento, e a função de combinação foi aprendida em vez de escrita.

A.2 Por que o gradiente desaparece ou explode ao longo de t

invocado no slide 04

O que se quer demonstrar. Que o gradiente que chega ao primeiro passo é um produto de $T-1$ jacobianas, e que a norma desse produto decai ou cresce exponencialmente em T .

ETAPA 1 • REGRA DA CADEIA NO TEMPO

$$\partial L / \partial h_1 = (\partial L / \partial h_T) \cdot \prod_{t=2..T} \partial h_t / \partial h_{t-1}$$

São $T-1$ fatores.

ETAPA 2 • A JACOBIANA DE UM PASSO

$$\partial h_t / \partial h_{t-1} = \text{diag}(1 - \tanh^2(a_t)) \cdot W_h = D_t W_h$$

D_t é diagonal com entradas em $(0,1]$ — só sabe encolher. E W_h é a **mesma** em todos os fatores.

ETAPA 3 • A NORMA DO PRODUTO

$$\begin{aligned} \|\prod D_t W_h\| &\leq \prod \|D_t\| \cdot \|W_h\| \leq (\gamma \cdot \sigma_{\max}(W_h))^{(T-1)} \\ \rho &= \gamma \cdot \sigma_{\max}(W_h) \rightarrow \|\partial L / \partial h_1\| \leq \|\partial L / \partial h_T\| \cdot \rho^{(T-1)} \end{aligned}$$

$\rho < 1$ • a cota vai a zero: o gradiente **desaparece**; o sinal que chega ao primeiro token é indistinguível de ruído.

$\rho > 1$ • a cota deixa de garantir estabilidade. Não prova explosão: ela exige direção persistentemente expansiva.

NÚMEROS DE REFERÊNCIA

$$\begin{aligned} 0,9^{10} &\approx 3,49 \cdot 10^{-1} & 1,1^{10} &\approx 2,59 \\ 0,9^{50} &\approx 5,15 \cdot 10^{-3} & 1,1^{50} &\approx 1,17 \cdot 10^2 \\ 0,9^{100} &\approx 2,66 \cdot 10^{-5} & 1,1^{100} &\approx 1,38 \cdot 10^4 \end{aligned}$$

Dez por cento de perda por passo é irrelevante em dez passos e é aniquilação em cem.

Por que aumentar só a dimensão não resolve. O expoente da cota é $T-1$. $\sigma_{\max}(W_h)$ pode mudar com dimensão e inicialização; o ponto é que, mantido o ganho efetivo por passo, aumentar d_h não remove o expoente nem encurta o caminho. É capacidade contra otimização — e a solução (A.3) mexe no caminho.

Recorte de gradiente. Renormaliza acima de um limiar: trata a explosão. Não há operação simétrica para o desaparecimento — amplificar um gradiente nulo amplifica ruído.

CASOS-LIMITE

T pequeno (3-5) $\rightarrow \rho^{(T-1)} \approx 1$: RNNs funcionam em janelas curtas.

$\rho = 1$ exato $\rightarrow$ cota constante, equilíbrio instável.

$|a_t|$ grande $\rightarrow \tanh' \rightarrow 0$: saturar acelera o desaparecimento.

Intuição. Iterar a mesma transformação alinha o vetor com o autoespaço dominante e escala por λ_1 a cada passo — juros compostos.

A.3 O gating do LSTM e o caminho aditivo

invocado no slide 06

AS SEIS EQUAÇÕES, COMPLETAS

```
f_t = σ(W_f [h_(t-1), x_t] + b_f)
i_t = σ(W_i [h_(t-1), x_t] + b_i)
o_t = σ(W_o [h_(t-1), x_t] + b_o)
g_t = tanh(W_g [h_(t-1), x_t] + b_g)
c_t = f_t ⊙ c_(t-1) + i_t ⊙ g_t
h_t = o_t ⊙ tanh(c_t)
```

Sigmoide nas portas porque a porta é uma fração de passagem — (0,1); tanh no candidato porque um delta precisa de sinal — (-1,1). Parâmetros: $4 \cdot [d_h(d_h + d_x) + d_h]$. O gating não é grátis em parâmetros; é grátis em estabilidade.

DERIVAÇÃO DO CAMINHO DA CÉLULA

```
∂c_t/∂c_(t-1) = diag(f_t) + [termos via h_(t-1)]
∂c_T/∂c_1 ≈ Π diag(f_t) = diag( Π f_t )
```

Premissa: $\text{diag}(f_t)$ é o caminho **direto** e domina no regime de saturação das portas — o regime em que a memória é preservada. Fora dele, o LSTM se comporta mais como uma RNN.

TRÊS CONSEQUÊNCIAS

- 1 Nenhuma multiplicação por matriz no caminho: nenhum valor singular entra na conta.
- 2 Se $f_t \approx 1$, o produto é ≈ 1 e o gradiente atravessa T passos sem atenuação. **A rodovia aditiva é isto.**
- 3 Se $f_t = 0,9$, o produto volta a $0,9^{(T-1)}$ — só que agora **por decisão da rede**, e apenas nos canais que ela escolheu esquecer.

Truque prático. Inicializar $b_f = 1$ faz $f_t \approx \sigma(1) \approx 0,73$ já no primeiro passo. Com $f = 0,5$, $0,5^{20} \approx 10^{-6}$: a memória morre em vinte passos antes de o treino começar.

CASOS-LIMITE

$f = 1, i = 0 \rightarrow c_t = c_{(t-1)}$: registrador — memória perfeita, entrada ignorada.
 $f = 0, i = 1 \rightarrow c_t = g_t$: célula reescrita; degenera em memória de um passo.
 $o = 0 \rightarrow h_t = 0$: a célula segue sendo atualizada e nada sai. É propriedade, não bug.

Leitura. O LSTM não elimina o desaparecimento: tira-o das mãos da álgebra e o põe nas mãos do modelo, que pode manter a porta aberta nos canais onde a dependência longa importa.

A.4 Por que um vetor de tamanho fixo degrada com o comprimento

invocado no slide
08

Premissas. O decoder recebe **apenas** c (definição do seq2seq sem atenção) e cada componente de c tem b bits úteis. Para o argumento de contagem supõe-se distinguir todas as entradas — é cota de informação, não premissa realista sobre tradução.

ARGUMENTO 1 · CONTAGEM (O TETO DURO)

```
log2(|V|^n) = n · log2|V| bits necessários  
d · b bits disponíveis na ponte  
n* = (d · b) / log2|V|
```

Para $n > n^*$, duas sequências distintas **necessariamente** recebem o mesmo c quantizado (casa dos pombos). Prova que a ponte não é injetiva — não prova erro de tradução.

```
d = 1000, b = 8, |V| = 32000 → log2|V| ≈ 14,97  
n* ≈ 8000 / 14,97 ≈ 534 tokens
```

Cuidado: é experimento mental sensível ao valor arbitrário de b . Mostra que capacidade finita existe; **não explica a curva** da Figura 2.

ARGUMENTO 2 · COMPRIMENTO DO CAMINHO

```
j → j+1 → ... → n (n - j passos no encoder)  
c  
s_1 → s_2 → ... → s_t (t passos no decoder)  
comprimento total = (n - j) + t
```

Cresce com n , e cada passo é um fator no produto de A.2. Aumentar d desloca n^* linearmente e não muda um único termo desta soma — **é por isso que a curva se desloca sem mudar de forma.**

Leitura. Faltam duas coisas diferentes: **espaço** (argumento 1) e **acesso** (argumento 2). Aumentar d compra espaço. A atenção (A.5) não aumenta d : torna o comprimento do caminho igual a **1**, para qualquer j e qualquer t .

CASOS-LIMITE

$n = 1 \rightarrow c = h_1$: nenhuma compressão, nenhuma perda — coerente com a curva plana em frases curtas.

$n \rightarrow \infty$ com d fixo $\rightarrow$ códigos compartilhados; o efeito depende de qual informação a tarefa precisa preservar.

$d \rightarrow \infty$ com n fixo $\rightarrow$ o argumento 1 deixa de limitar, mas o caminho recorrente continua longo: dificuldade de otimização, não erro inevitável.

A.5 O alinhamento de Bahdanau, derivado

invocado no slide 10

AS QUATRO ETAPAS

```
e_(t,j) = v^T tanh(W s_(t-1) + U h_j)
a_(t,j) = exp(e_(t,j)) / Σ_k exp(e_(t,k))
c_t = Σ_j a_(t,j) h_j
s_t = f(s_(t-1), y_(t-1), c_t)
p(y_t | y_(<t), x) = g(y_(t-1), s_t, c_t)
```

O score é uma MLP de uma camada oculta com d_a unidades, **compartilhada por todas as posições j** — é o compartilhamento que permite frases de qualquer comprimento. Dimensões: $W s \in \mathbb{R}^{d_a}$, $U h \in \mathbb{R}^{d_a}$, v^T colapsa num escalar.

EXEMPLO NUMÉRICO COMPLETO

```
e = (2,0; 1,0; 0,5) → Σ exp = 11,7561
a = (0,6285; 0,2312; 0,1402) soma = 1,0000
e dobrado = (4,0; 2,0; 1,0)
a = (0,8438; 0,1142; 0,0420)
```

$\alpha_1/\alpha_2 = \exp(e_1 - e_2)$: diferença de 1 no score vira fator 2,72 no peso. O máximo saltou de 0,63 para 0,84 **sem mudar a ordem** — o softmax amplifica, e a amplificação depende da **escala**. É o que motiva o $\sqrt{d_k}$ na Aula 6.

Normalização. $\alpha_{(t,j)} > 0$ e $\sum_j \alpha_{(t,j)} = 1$: cada passo do decoder recebe um orçamento unitário de leitura, num simplex. É restrição de normalização da linha — não um jogo de soma zero.

Combinação convexa. Como $\alpha \geq 0$ e soma 1, c_t é um ponto no fecho convexo da nuvem $\{h_j\}$. Não é seleção, é mistura — por isso a saída quase nunca coincide com um h_j isolado.

POR QUE É TREINÁVEL

$$\partial a_i / \partial e_j = a_i (\delta_{ij} - a_j)$$

Não nula para todo j com α_j longe de 0 e 1: o gradiente alcança **todos** os h_j . Um argmax rígido teria derivada zero quase em toda parte.

O comprimento de caminho, que é o ponto. Depois que h_j existe, há uma aresta direta de h_j a c_t via $\alpha_{(t,j)} h_j$. Ela **não elimina** os passos recorrentes que produziram h_j nem os do decoder.

CASOS-LIMITE

scores iguais $\rightarrow \alpha = 1/n$: média simples, “não sei onde olhar” ($\approx$ inicialização). um score domina $\rightarrow \alpha \rightarrow 1$: alinhamento duro, e o gradiente para os demais h_j desaparece.

$n = 1 \rightarrow \alpha = 1$: degenera no seq2seq clássico.

A.6 De Bahdanau a Q/K/V: a passagem formal

invocado no slide 12

O que se quer mostrar. As duas atenções pertencem à mesma família funcional: calcular relevâncias, normalizá-las, misturar valores. Uma não é caso particular exato da outra — mudam o score, as projeções, a origem das consultas e a forma de executar.

```
FORMA GENÉRICA
Attention(q,{k_j},{v_j}) = Σ_j softmax_j(score(q,k_j))·v_j

TRANSFORMER
Q = X_q W_Q · K = X_k W_K · V = X_v W_V
Attention(Q,K,V) = softmax(QK^T/√d_k)V
```

Projeções explícitas. Aprender W_Q, W_K, W_V separadamente permite que uma posição se anuncie por uma característica e entregue outra. Bahdanau já separa parcialmente (U_h h_j no score, h_j na soma); 2017 torna a separação simétrica e matricial.

Por que √d_k. Se q e k têm componentes independentes, média 0 e variância 1, Var(q·k) = d_k; o desvio cresce como √d_k. Dividir devolve os scores à ordem de grandeza 1 e evita saturação precoce do softmax.

Retirar a recorrência. Q/K/V da mesma sequência define self-attention; isso, sozinho, não dispensa RNNs. O Transformer decide substituir camadas recorrentes por blocos de self-attention e feed-forward. A geração autorregressiva continua sequencial.

PAPEL	BAHDANAU 2014	TRANSF. 2017	CONSEQUÊNCIA
Query	q_t = W_s s_(t-1)	Q = X_q W_Q	uma consulta x todas, em matriz
Key	k_j = U_h h_j	K = X_k W_K	projeção explícita
Value	v_j = h_j	V = X_v W_V	conteúdo com projeção própria
Score	v_a^T tanh(q_t+k_j)	QK^T/√d_k	produto denso, escala controlada
Normaliz.	softmax sobre j	softmax por linha	pesos positivos somam 1
Agregação	Σ_j α_(t,j) v_j	A V	mesma operação, forma matricial
Estados	encoder/decoder recorrentes	blocos sem recorrência	paralelismo vem da arquitetura, não da palavra “atenção”

CASOS-LIMITE

W_Q = W_K = W_V = I → softmax(XX^T/√d)X: há mistura contextual sem projeções aprendidas, mas sem codificação posicional o mecanismo é equivariante a permutação. n = 1 → matriz 1x1 igual a 1: a saída é o próprio value. W_K = W_V → chave e valor compartilham projeção; reduz a separação de papéis e não recupera Bahdanau, pois o score segue sendo produto interno.



Obrigado. Até a Aula 6.

Nome e Sobrenome

CESAR School

seuemail@cesar.org.br

cesar.org.br

MATRIZ

Recife – PE

COMERCIAL

São Paulo – SP

INTERNACIONAL

Flórida – EUA

REGIONAL

Manaus – AM

COMERCIAL

Rio de Janeiro – RJ

INTERNACIONAL

Aveiro – PT